

AI4DB 连接优化策略研究综述

摘要：连接优化是数据库系统中非常重要但也很难解决的问题，直接影响查询的执行速度。传统优化器主要使用启发式规则 and 成本模型，但在处理复杂查询时存在基数估计误差会累积、优化策略不够灵活、无法利用历史执行信息等问题。近年来，深度学习和强化学习在连接优化方面取得了不少进展。本文整理了 2022 年到 2025 年期间的相关研究，主要从基数估计、连接顺序选择和端到端优化器三个方面来介绍，总结了这些方法的核心思路、实现方式和实验结果，并针对 OLTP、OLAP、HTAP 等不同应用场景给出了方法选择的建议。最后讨论了未来可能的研究方向，包括如何提高训练效率、增强泛化能力、提高可解释性等。

关键词：数据库优化；连接顺序选择；深度强化学习；基数估计；学习型优化器

一、引言

连接操作是关系数据库中最基本但也是最耗时的操作。在实际的数据分析系统中，查询经常需要连接多张表，连接顺序的选择会直接影响查询的执行速度。研究发现，对于涉及多张表的复杂查询，不同的连接顺序执行时间可能相差很大，在 Join Order Benchmark 测试集中可达 1000 倍以上。

然而，找到最优连接顺序本身是一个 NP-hard 问题。对于包含 n 张表的查询，可能的连接顺序数量随 n 呈指数增长：左深树结构有 $O(n!)$ 种可能，而 bushy 树结构更可达到 $O(4^n)$ 种。这种组合爆炸使得枚举所有方案变得不可行，特别是当查询涉及 10 张表以上时。

传统查询优化器主要有三个部分：基数估计器、成本模型和计划枚举器。虽然传统优化器已经发展了几十年，但在处理复杂查询时仍存在问题。基数估计是查询优化的基础，传统方法一般假设不同列之间是独立的，用直方图、采样等统计方法来估计。但实际数据往往有复杂的相关性和偏斜分布，基数估计的误差会在连接树中向上传播并不断放大，单表谓词的 α 倍估计误差可能导致整个计划成本误差达到 α^4 倍。传统优化器使用静态的启发式规则和成本模型，包含大量针对特定硬件和数据分布调整的“魔法常数”，当数据分布变化或运行环境改变时，这些固定参数可能失效，优化器无法从经验中学习。对于简单查询，System R 的动态规划算法通常能找到较好的解，但遇到包含环路或团结构的复杂查询图，特别是数据分布高度偏斜时，启发式规则经常失效。另外，传统优化器每次查询独立优化，不会记录历史执行情况，无法利用历史信息改进决策。

近年来，深度学习和强化学习在计算机视觉、自然语言处理等领域取得了很大进展，促使研究者探索将 AI 技术应用于数据库系统，形成了“AI for Database”研究方向。把 AI 技术用到连接优化上既有必要，也具备可行性。机器学习模型可以从历史查询执行数据中学习，发现传统启发式难以捕捉的模式，实现基于数据的自适应优化。与传统方法分别优化中间目标不同，深度强化学习可以直接以最终查询执行时间作为优化目标，避免中间环节误差累积。强化学习的试错机制让优化器通过尝试不同计划选择，从成功和失败经验中不断学习。GPU 等加速硬件的普及、深度学习框架的成熟以及标准化基准测试的建立，为学习型优化器提供了技术基础。ReJOIN[8]、Neo[12]、Bao[13]等早期工作已证明学习型优化器在某些场景下能够明显超过传统方法[1]。

连接优化问题可描述为：给定一个涉及 n 张表的 SPJ 查询和一组连接谓词，连接顺序选择的任务是在所有合法顺序中选择一个，使最终计划的执行时间最短。合法性通常要求避免笛卡尔积，只有通过连接谓词相连的表才能进行连接。查询可表示为无向图，节点是表，边代表连接谓词，图的拓扑结构直接影响优化难度。一个连接顺序对应一棵二叉树，根据树的形状可分为左深树、右深树、之字形树和 Bushy 树，其中 Bushy 树的搜索空间最大。在连接

树的每个内部节点，还需为连接操作选择物理算子：嵌套循环连接、哈希连接或归并连接。物理算子的选择和连接顺序相互影响，同时优化两者虽然复杂度更高，但往往是充分利用硬件性能的唯一办法。端到端学习型优化器想要统一优化所有决策，但这大大增加了问题复杂度。连接顺序的搜索空间随表数量呈超指数增长，即使采用 System R 的动态规划剪枝，复杂度仍高达 $O(3^n)$ ，对于 15 张表以上的查询仍无法穷举。

本文主要关注 SPJ 查询，不涉及嵌套子查询、相关子查询、聚合操作、集合操作和窗口函数。这个限制是因为现在大多数学习型优化器还不支持这些复杂特性，但 SPJ 查询已覆盖数据仓库和分析系统中的很多实际应用场景。本文从三个角度对 AI4DB 连接优化技术进行分类：基数估计优化、连接顺序选择优化和端到端优化器。

二、技术分类与核心方法

基数估计是查询优化的基础，估计准确性直接影响后续的成本估算和计划选择。传统方法基于独立性假设和统计信息，而 AI 方法通过学习捕捉数据中的相关性和模式。FactorJoin[2] 用因子图建模连接查询中的概率依赖关系，将基数估计问题转化为因子图上的变量消除问题。它分为离线准备和在线推理两个阶段：离线阶段构建单表条件分布，使用 GBSA 算法对等价键分组做联合分箱，通过 Chow-Liu 结构学习将多列联合分布近似为树状依赖；在线阶段将查询图转为因子图，复用已训练的单表估计器，按分箱域运行近似变量消除，直接给出上界估计。FactorJoin 适合星型查询和链式查询，在 STATS-CEB 和 IMDB-JOB 基准上，相比 FLAT 等模型，估计延迟低 40 倍、模型体积小 100 倍、训练时间快 100 倍，在 JOB 基准测试上达到中位数 Q-error 为 1.82，明显优于 PostgreSQL 的默认估计器。

ALECE[3] 针对动态数据环境设计，采用双模块注意力架构实现增量式状态更新。Data-Encoder 模块用自注意力机制编码数据库的当前状态，通过 dx-bin 直方图表示，每个属性维护固定大小的直方图，支持 $O(1)$ 复杂度的增量更新。Query-Analyzer 模块用交叉注意力机制分析查询与数据库状态的交互。当数据插入或删除时，ALECE 只需更新相关属性的 dx-bin 直方图，无需重新训练整个模型，特别适合 HTAP 系统。在 Insert-heavy、Update-heavy 和 Distribution-shift 三类混合负载上，ALECE 对 JOB-light、STATS、TPC-H 等数据集的端到端查询时间提升最高达到 2.7 倍，数据变化后仍能保持 Q-error 小于 1.6，推理延迟小于 11 毫秒，比需要重新枚举特征的 Fanout 方法快 3.4 倍。

连接顺序选择是查询优化的核心问题，基于深度强化学习的方法将其建模为序列决策问题，通过与数据库环境的交互学习最优策略。价值函数方法学习价值函数评估状态或状态-动作对的质量，然后通过贪婪策略选择动作。DQ[9] 是第一个将 Deep Q-Network 用于连接顺序选择的工作，但简单的 One-Hot 编码无法有效捕捉查询的结构信息。RTOS[10] 采用 Tree-LSTM 编码连接树的结构信息，在 JOB 上训练 7.2 小时后 GMRL 达到 0.65。JOGGER[11] 采用图卷积网络降低模型参数量，并引入课程学习加速训练，在 JOB 上达到 SOTA 水平，训练时间 7.2 小时，模型参数量比 RTOS 减少 40%。

策略梯度方法直接学习策略，输出在某个状态下选择某个动作的概率。ReJOIN[8] 是第一个将 PPO 用于连接顺序选择的工作，但向量表示信息损失严重，难以处理复杂查询。AlphaJoin[17] 借鉴 AlphaGo，采用 MCTS 进行计划搜索，但每次查询需要多次模拟，推理时间较长。在线学习方法中，SkinnerDB[16] 在查询执行过程中在线优化连接顺序，不需要预训练，采用 UCT 算法平衡探索与利用，提供理论上的后悔界限，但需要定制执行引擎支持中途切换计划，集成难度较大。

Lero[5] 采用成对比较的学习排序方法，核心观察是计划的相对排序比绝对成本预测更重要。模型输入是两个计划的编码，输出表示一个计划优于另一个计划的概率。通过基数调优策略设定缩放因子集合，对每个子查询的基数估计乘以不同的缩放因子生成多样化的候选计划。在合成工作负载上预训练，使用 PostgreSQL 的估计成本作为标签，预训练时间仅需 5

分钟即可快速继承传统优化器的知识。在 IMDB 数据集上达到 2.87 倍加速，GMRL 为 0.43，在 STATS 数据集上减少延迟 44%，尾部性能很好，最大减速只有 500 毫秒。

端到端优化器不仅选择连接顺序，还决定物理算子、访问路径等，目标是生成完整的执行计划。Neo[12]是第一个端到端学习型优化器，使用深度神经网络替代传统的成本模型和枚举器，采用 Tree-CNN 编码计划树预测执行延迟，但需要大量专家示范数据，推理时间较长。Balsa[14]通过模拟器 bootstrap，不需要专家示范，基于成本模型的简化执行引擎快速生成大量训练数据，引入约束优化避免生成灾难性计划，在 JOB-RS 上达到 1.59 倍加速，但在 Stack Overflow 上训练失败。

混合型优化器结合了传统优化器的稳定性和学习型方法的性能提升潜力。Bao[13]不重建优化器，而是学习选择最优的全局提示集，采用 Thompson 采样把问题建模为上下文多臂老虎机，集成简单，只需要加载预训练模型和配置 pg_hint_plan，在 JOB 上训练 6.2 小时，容易回滚，但提示集数量有限限制了性能提升空间。HybridQO[15]利用传统成本模型生成候选计划，用 MCTS 选择最优计划，为每个计划估计不确定性，拒绝高不确定性的计划，传统优化器作为兜底提供稳定性保证，但 MCTS 推理开销较大。

LOGGER[4]是目前综合性能最好的方法之一，采用 Graph Transformer 做表示学习，通过注意力机制和 Laplacian 位置编码捕捉全局结构信息。ROSS 是 LOGGER 的关键创新，平衡了学习型优化器与传统优化器的优势，学习型优化器选择限制算子，DBMS 优化器在限制下选择具体物理算子。 ϵ -beam 搜索保持多条搜索路径，自适应探索概率根据查询性能动态调整。奖励加权与对数变换处理延迟范围跨越 5 个数量级的问题，通过算子相关延迟和算子无关延迟的加权降低不良算子对后续决策的影响。在 JOB 测试集上 WRL 为 0.48，训练时间 23.2 小时，尾部性能很好。

表示学习的演化反映了这个领域的技术进步。One-Hot 编码实现简单但信息损失严重。Tree-LSTM 把 LSTM 扩展到树结构，能捕获树的结构信息和层次关系，但缺乏全局视野。Tree-CNN 使用卷积核在树上滑动，计算效率高，容易并行化，但感受野受限。GCN 在查询图和模式图上应用图卷积网络，参数量明显减少，但存在过平滑问题。Graph Transformer 结合 Transformer 的注意力机制和图结构的位置编码，通过多头注意力捕获全局依赖关系，但计算复杂度是平方级，大图上开销大。注意力机制用于捕捉不同属性间的相关性和查询与数据状态的交互，但需要大量训练数据。

搜索策略决定了如何在庞大的计划空间中找到高质量的解。 ϵ -greedy 以固定概率随机选择动作，实现简单但随机探索效率低。 ϵ -beam search 保持多条搜索路径，引导探索从预测较优的候选中随机选择，自适应探索概率根据查询性能动态调整。Thompson 采样采用贝叶斯优化，维护每个提示集的后验分布，提供理论保证的后悔界限，自动平衡探索与利用。MCTS 通过选择、扩展、模拟、回传四个阶段在一棵树中模拟多个可能路径，但每次查询需要多次模拟，推理时间长。UCT 采用 UCB 公式显式平衡探索与利用，提供理论上的后悔界限，适用于在线决策。Best-first beam search 贪婪选择 top-k 候选，收敛快但容易陷入局部最优。

三、优势分析

学习型优化器在查询执行延迟上取得了明显的性能提升。在 JOB 基准测试上，LOGGER 的 GMRL 达到 0.66，Lero 达到 0.43-0.66，RTOS[10]达到 0.65，JOGGER[11]达到 SOTA 水平。FactorJoin 在包含外键连接的查询子集上达到中位数 Q-error 为 1.82，明显优于 PostgreSQL 的默认估计器，估计准确性提升了超过 5 倍。性能提升的根本原因在于消除了基数估计误差的累积效应，学习型方法通过因子图精确建模依赖关系、直接优化计划排序、奖励加权降低算子选择错误的影响等方式缓解问题。学习型方法能够捕捉数据分布中的复杂模式，深度学习模型能学习传统启发式难以表达的非线性模式。另外，学习型优化器能够从历史执行反馈

中学习，持续学习识别和修正系统性错误，而传统优化器每次查询独立优化，无法利用历史信息。

尽管学习型优化器需要额外的模型推理时间，但相对于查询执行时间，这一开销通常很小。LOGGER 的推理时间最短为 53 毫秒。学习型方法通过 Beam Search、 ϵ -beam Search、Thompson Sampling、成对比较等策略高效搜索，将复杂度从 $O(3^n)$ 降至 $O(K \cdot n)$ 或 $O(n^2)$ ，支持并行优化多个查询。

在鲁棒性方面，学习型优化器通过压缩灾难性计划、加权修正、不确定拒绝、理论保证等机制，把尾部风险控制在可接受的范围。LOGGER 用对数变换把千倍慢计划的奖励压缩两个数量级，再用算子相关和无关延迟加权，实际测试中最大回退只有 11%，最大加速达到 10.3 倍。Lero 通过预训练继承 PostgreSQL 的底线，最大回退降到 5.2%，平均仍保持 2.87 倍提速。HybridQO[15]引入不确定性感知，直接拒绝方差高的计划，回退概率再降低 30%。

自适应能力是学习型优化器的核心优势，体现在对数据分布变化和工作负载变化的动态适应。ALECE 采用 dx-bin 直方图维护数据库状态，支持 $O(1)$ 复杂度的增量更新，数据插入或删除时只更新相关属性的直方图，不需要重新训练整个模型。Lero 的成对比较模型对数据变化更鲁棒，计划的相对顺序比绝对延迟更稳定，每 100 个查询更新一次模型。LOGGER 通过奖励加权把 90% 的信号固定在连接顺序上，只剩下 10% 随算子波动，这样连接顺序的知识能保持稳定，算子选择能快速调整。面对工作负载变化，POLAR[6]在运行时通过评估不同连接路径的执行效果来实时重选连接路径，不需要离线重训练就能在负载切换后收敛到新的最优解。Bao[13]用 Thompson 采样持续更新提示集的后验分布，自动降低失效 hint 的选中概率。

可解释性决定了 DBA 能否信任并调试优化决策。FactorJoin 把基数估计拆成因子图上的变量消除，每一步都可以逐步验算，误差来源很清楚。Lero 把成对比较结果绘成偏序图，通过梯度反查关键特征，使决策具备可追溯的推理链。LOGGER 的 ROSS 接口把决策拆成先禁用算子、再交给原生优化器两步，配合 Graph Transformer 注意力热图，DBA 可以直观看到是哪几张表的全局关联促使模型做出决策。ALECE 的交叉注意力则高亮查询谓词中对行数影响最大的属性，指导 DBA 有针对性地更新统计信息或补建索引。

非侵入式集成把能否先跑起来放在优先级首位。最轻量的方案只依赖 pg_hint_plan，Bao[13]、COOOL[18]用 6.2 小时训练就能给出 1.7 倍提速，关闭 hint 立即回滚。Lero 再增加一个成对比较钩子，仍然不需要内核改动，平均加速 2.87 倍。LOGGER 的 ROSS 接口把禁用某类算子下推到计划生成阶段，只需要局部微调约 2000 行代码。POLAR 在运行时插入 Multiplexer 与采样算子，约 3500 行代码就能让 DuckDB 获得 3 倍尾部延迟削减。Neo[12]和 Balsa[14]则必须重建整套优化器，代码量增加到万级，冷启动与维护成本最高。

泛化能力决定学习型优化器能否举一反三。JOGGER[11]用课程学习按表数从少到多渐进训练，使同一套参数在不同表数查询上都能保持低误差。LOGGER 的 Graph Transformer 通过拉普拉斯位置编码和多头注意力，把全局表间关系统一编码，跨查询模板不需要重训练就能在新样例上取得与训练集相当的 GMRL。面对模式变更，LOGGER 只需要把新表节点插入已有图结构并微调 4.6 小时，原表嵌入通过共享权重保留。同一套模型在 PostgreSQL 上完成训练后，可以直接迁移到兼容协议的商业库，仍能维持 1.8 倍平均提速。

四、不足与挑战

尽管学习型优化器在性能和技术上有很多优势，但它们在实际应用中还是面临不少挑战。训练时间已经成为学习型优化器走出实验室的首要瓶颈。以 LOGGER 为例，在 JOB 负载上完成一次完整训练需要 23.2 小时，其中 60% 花在执行候选计划上。当连接数增加到 10 以上时，搜索空间从千万级跳到十亿级，需要执行的候选计划数量随之倍增，训练周期可能被拉长到数日甚至一周。深度强化学习还需要为奖励函数、网络结构、探索率等大量超参数寻找最优组合，LOGGER 使用 Optuna 调参就额外耗费 3 天。课程学习、预训练与蒸馏等策略可以部分

缓解，但还无法在大规模真实负载上实现数量级加速。

数据成本是学习型优化器面临的第二个大问题。离线方案如 ALECE 需要至少一万条查询-状态-真实基数的三元组，生产环境往往要采集数周到数月，而且新系统冷启动没有日志、脱敏又可能洗掉关键特征。Neo[12]这类专家示范方法更要求 DBA 先给出高质量计划，人工标注代价很高。Balsa[14]用模拟器替代人工，却引入了成本模型失真的风险，在 Stack Overflow 上训练 60 小时仍不收敛就是例子。SkinnerDB[16]虽然宣称零数据启动，却要求执行引擎能在跑查询中途换计划，标准 DBMS 内核没有这个能力，集成难度很大。

计算资源门槛把跑实验变成了烧预算。LOGGER 在单张 RTX 2080Ti 上训练 23.2 小时就消耗约 200 美元云租金，如果再算上三天超参数调优，总成本接近 400 美元。除了 GPU，Graph Transformer 的 50M 参数、2GB 级激活与经验回放缓冲区合计需要 2.5GB 显存加 8GB 系统内存，实验迭代一多，本地工作站经常显存告急。存储方面，每轮训练要保存查询特征、执行延迟、模型快照与日志，TB 级磁盘随工作负载倍增迅速膨胀。

泛化能力不足是学习型优化器面临的核心挑战之一。以 TPC-DS 为代表的多渠道零售决策场景，把 23/99 条模板写成嵌套子查询，并在同一语句内包含窗口函数、ROLLUP、CUBE、UNION 等复杂构造，使现有主要面向 SPJ 的学习型优化器瞬间失效。即使退一步只看连接图，其拓扑敏感性也足以让模型水土不服：星型或链式连接存在局部最优贪心策略，模型容易快速收敛；而环型、团型结构需要全局权衡，搜索空间急剧增加，LOGGER 的消融实验显示，同样参数的网络在环或团查询上的 GMRL 比星型下降 35% 以上。更严重的是谓词选择性分布漂移，训练集如果集中在高选择性点查，测试时遇到低选择性范围扫描，估计值可能偏差一个数量级。深度模型很容易过拟合训练工作负载的特定模式，经典正则化手段只能把测试集 WRL 从 0.71 拉到 0.69。

表数量扩展性决定学习型优化器能否从小见大。现有实验一致表明：所有方法在内插区间表现稳健，一旦进入外推性能就明显下滑，其中 DQ[9]因为 One-Hot 表示容量不足在外推场景几乎完全失效。当连接规模增加到 10 表以上时，搜索空间与模型容量双重瓶颈凸显：左深树 3.6M、Bushy 树 1.7×10^8 ，再乘以物理算子组合，可供探索的计划瞬间破百亿。Tree-LSTM 只有 73k 参数、Tree-CNN 感受野受限，都难以刻画如此高维模式，即使 Graph Transformer 把参数量提高到 8.5M，仍然远低于 NLP 领域百亿级模型的容量。更棘手的是训练数据稀少，JOB 中 10+表查询不足 5 条，远不足以覆盖空间，导致深层网络很容易过拟合。

跨 DBMS 迁移要把在 A 系统训练出的模型搬到 B 系统，却遇到三个问题：成本模型、系统参数与接口兼容性。PostgreSQL 与 MySQL 对哈希连接的定价公式里，魔法常数不同，内存成本显隐式建模也不同，导致同样计划在两边的延迟标签天然错位。LOGGER 的实验表明，直接把 PostgreSQL 上训练的 Graph Transformer 搬到商业兼容库，GMRL 从 0.66 微升到 0.71，但仍需要 2 小时在线微调才能回到 0.68。参数层面，work_mem 从 64MB 调到 256MB 后，传统成本模型会立即下调哈希连接价格，而延迟导向的神经网络对此没有反应，必须重新标注样本、重新训练或增量微调。接口差异进一步抬高门槛，不同 DBMS 的扩展接口差异很大，跨 DBMS 迁移仍需要针对成本公式、参数敏感性与接口能力做一库一策的二次适配。

鲁棒性问题直接影响学习型优化器在生产环境中的可靠性。灾难性计划是投产的红线，一次超时就可能拖垮整个集群。现有四条防线都治标不治本：Timeout 阈值设得太短会误杀正常慢查询，设得太长又拦不住真正的灾难；对数变换虽然把极端奖励压缩两个数量级，却同时稀释了差与灾难的区分度；硬约束把探索空间直接砍掉一半，安全性提升的同时也把潜在提速计划拒之门外；不确定性感知需要多次模拟，推理时间翻倍，而且对未见查询类型几乎无法估计不确定度。冷启动是上线前的信心关，模型参数随机初始化时输出接近均匀分布，前 50 个 Epoch 里 90% 动作靠 ϵ -贪婪随机选出，GMRL 一度跌到 1.4 倍，劣于原生 PostgreSQL。为了把学习成本降到最低，预训练和专家示范成为两条主流捷径，Lero 在合成负载上用成本

模型生成 5 万条计划做 pairwise 预训练，仅 5 分钟就达到与 PostgreSQL 持平的 0.98 GMRL。

奖励稀疏与高方差把连接优化变成盲人走钢丝，只有整棵连接树执行完才拿到唯一延迟，中间任何局部动作都没有即时反馈；同一顺序下换一套物理算子，延迟可能差百倍，导致 Q 值标准差飙到 13.4 秒，模型难以收敛。LOGGER 用奖励加权把总延迟拆成顺序相关和算子相关两段，赋予顺序 90%权重、算子仅 10%，既让连接骨架获得稳定信号，又允许算子层做小幅探索，价值网络方差下降 42%，训练曲线提前 30 个 epoch 收敛。

实用性问题关乎学习型优化器在实际生产环境中的可部署性和可维护性。现实 SQL 的复杂程度远超实验室里的单表-连接模板，TPC-DS 里 23%模板含嵌套子查询，电商生产日志中这一比例更达到 40%，而当前主流学习型优化器仍普遍停留在 SPJ 层面，对子查询、聚合、窗口函数、集合运算等特性都不支持，导致在完整 TPC-DS 上几乎无法产生可执行计划。系统集成复杂度把跑通 Demo 与上线生产隔成两道天堑，Bao[13]凭借 pg_hint_plan 可以在一天内完成部署，而 Balsa[14]这类重写优化器路线，需要 10k 行代码、2-3 周开发调试与 200 小时年度维护。版本兼容性、性能监控与故障排查构成三大工程问题，模型依赖特定内核版本，升级就需要重训练；除查询延迟外，还需要持续追踪模型置信度、Q-error 与特征漂移；一旦性能回退，DBA 要同时检查计划、特征、模型权重和超参，跨界门槛很高。

可维护性是把能用变成长用的最后一道关卡。Lero 的在线更新策略每天跑两次，100 分钟乘以 365 天约 600 小时 CPU 时间，对高负载生产系统而言相当于持续占用一台核心节点。多工作负载并行场景下，版本管理更棘手，rollback 需要保留历史 checkpoint 并秒级切换，A/B 测试要同时加载多份模型并确保特征兼容。文档与知识传承同样棘手，DBA 不熟悉反向传播，ML 工程师看不懂索引路径，双方各说各话导致故障排查链路拉长。

五、适用场景分析

根据不同的应用场景和系统特征，学习型连接优化器的方法选择需要结合工作负载特征、系统成熟度和数据特征进行综合考虑。OLTP 场景的核心诉求是高并发、短查询、快响应，每秒上万次、2-4 表连接、P99 小于 100 毫秒，且数据持续更新。对此，Bao[13]成为零侵入首选，73 毫秒推理、pg_hint_plan 一键开关、回滚零成本，在模拟 OLTP 负载里仍能给出 1.3-1.6 倍提速。如果外键健全，可以再加上 FactorJoin 做基数估计，纯统计推理、无训练延迟、可解释，能把索引点查的 Q-error 从 10 降到 2 以内。反之，Neo[12]和 Balsa[14]的 160-188 毫秒推理远超 P99 预算，LOGGER 虽然只要 53 毫秒，但对 2-3 表查询传统优化器小于 5 毫秒已经足够，所以只建议在 OLTP 中偶发的复杂分析查询里启用。

OLAP 查询经常是 8-15 表连接、秒级到分钟级延迟，数据经 ETL 后相对稳定，对跑得快很敏感，可以接受分钟级训练成本。综合性能与鲁棒性，LOGGER 被验证为 OLAP 首选，JOB 上 2.07 倍加速、WRL 为 0.48、75%查询回退小于 10%，且已在 TPC-DS、Stack Overflow 等多负载复现，训练 23 小时一次即可长期受益。如果追求极致性能，可以换 Lero，2.87 倍提速、尾部最大减速只有 0.5 秒，且每 100 查询在线更新，5 分钟预训练即可冷启动。对于金融、风控等宁慢勿错场景，HybridQO[15]提供保守策略，MCTS 先用不确定性感知筛掉高风险计划，必要时直接回退 PostgreSQL，实验显示回退率 8%、整体仍提速 1.4 倍。

HTAP 在同一套数据上同时跑高并发短事务与长耗分析，实时更新与资源竞争让一招通吃成为奢谈。对此应该采用动态估计、运行时自适应、分层路由的混合方案：ALECE 以 O(1) 增量直方图持续跟踪数据变化，插入或删除后 Q-error 涨幅小于 10%，为短时查询提供即时可靠的基数信号；POLAR 在查询执行阶段通过采样与执行效果评估在线重排连接顺序，无需离线训练即可在负载变化后 30 秒内收敛到新最优，且只增加毫秒级采样开销。系统层再辅以查询路由，点查或小范围写走 OLTP 引擎，5 表以上复杂分析自动流入 OLAP 通道，两层间共享 ALECE 直方图，避免重复采样。

在数据湖或联邦查询场景下，数据呈分布式、多源、弱统计三大特征，S3、HDFS、

PostgreSQL、MySQL 等异构源通过 Trino 或 Presto 统一入口，网络延迟不稳定、部分源缺乏准确统计信息，导致传统成本模型频繁误判。对此应该采用轻量级、可解释、零历史依赖的组合方案。FactorJoin 作为首选基数估计引擎，只需要单表 count、直方图等最小统计，可在各源独立采集，不需要跨源导出数据或预跑查询，通过外键直方图和变量消除完成多表连接估计，训练与模型体积几乎为零，特别适合外键关系清晰的数据湖星型模式。ADOPT[7]辅助多路或环状连接，当查询出现环路或大型团状连接时，可以启用 ADOPT 的 Leapfrog TrieJoin 和 UCT 在线属性排序，避免中间结果爆炸，实际测试中环形查询比传统二步连接快 5-10 倍。

按系统成熟度分类，研究原型阶段目标在于探索性能上限、验证技术可行性，资源充足、时间不受严格 deadline 限制、风险承受度高，推荐尝试最新方法如 LOGER、Lero、POLAR。生产试点阶段目标在于验证实际收益、控制风险，资源有限、时间需要在 1-3 个月内看到效果、风险承受度中等，推荐 Bao[13]作为风险最低的首选方案，集成简单 1 天可完成部署，容易回滚关闭 pg_hint_plan 即可。如果团队有 GPU 资源且追求更高性能，可以尝试 LOGER，但必须采用影子模式，前 100 epoch 只做影子模式不实际执行学习计划，避免冷启动影响生产。大规模部署阶段目标在于全量覆盖、稳定运行，需要完善的基础设施、长期运维、风险承受度低，推荐 HybridQO[15]混合方案，传统优化器与学习型方法结合，引入熔断器机制，一旦发现延迟超过阈值立即回退到传统优化器。

按数据特征分类，高偏斜数据场景下数据分布极不均匀，热点数据集中在少数值，传统独立性假设失效严重。ALECE 的注意力机制能够捕捉属性间的相关性，在 IMDB 高偏斜数据上 Q-error 显著优于 PostgreSQL。Lero 的基数调优策略对每个子查询尝试不同的基数缩放因子，成对比较对比不同基数假设下的计划质量，即使基数估计不准，仍能选择相对更好的计划。FactorJoin 显式建模相关性，通过因子图表示属性间的依赖，可解释性强。

频繁更新数据场景下，高频 INSERT、UPDATE、DELETE 操作使数据分布快速变化，统计信息容易过期，学习型模型需要适应变化。ALECE 的 dx-bin 直方图固定大小、O(1)更新，无需重训练，数据变化后模型仍可用，实验验证在动态数据场景下优于 RTOS[10]和 JOGGER[11]。Lero 的持续学习框架在线更新，每 100 查询自动更新，标签稳定性使计划的相对顺序比绝对延迟更稳定。不推荐方案包括需要完全重训练的 DeepDB 和 FLAT，一旦不重训，Q-error 直接飙升到 10 以上。

稀疏连接图场景下表间连接谓词缺失，容易产生笛卡尔积，中间结果爆炸风险，传统优化器通常避免但有时是必要的。SkinnerDB[16]的 UCT 算法平衡探索与利用，自动发现安全路径，Regret Bound 理论保证在探索足够后能找到近优解，在线学习无需预训练，自动适应查询特征。System R 策略的基本原则是避免无条件连接，只有通过连接谓词相连的表才能连接，如果必须产生笛卡尔积，尽量延后，优先连接选择性高的谓词。

六、研究展望

AI4DB 连接优化领域仍面临诸多开放性科学问题，未来研究可从多个方向展开。训练效率提升是首要任务，主动学习可通过筛选最有信息量的查询减少训练数据量，迁移学习将预训练和微调范式推向跨库、跨负载，知识蒸馏可将模型压缩到更小规模，有望把单次训练成本从天级压到小时级。鲁棒性增强需要在模型上线前设置多层安全网，用传统成本模型过滤灾难计划，用不确定性门控拒绝高风险候选，在执行层实时比对并在线微调，可降低灾难性计划的发生率。可解释性研究要让 DBA 像阅读执行计划一样看懂模型决策，通过决策路径可视化和特征重要性分析。

大语言模型有望重塑查询优化的前端链路，LLM 可担任查询理解器，利用少样本与跨领域知识对新查询模板做出合理推断，同时用自然语言解释决策逻辑，天然提升可解释性。联邦学习可打破数据孤岛，各企业在本地训练子模型，仅上传经差分隐私处理的梯度，中央服务器聚合更新全局模型，预计 10 家企业联合即可等效获得 10 倍数据量。多模态数据将打破

行列值单一维度，需要处理向量相似度连接与关系连接的混合优化，通过成本-精度分层模型统一纳入代价方程。

放眼更长远的未来，可微分数据库、查询优化基础模型与自演化优化器将构成终极愿景：先把连接、排序等离散算子改写为可导形式，使数据库能在每次查询后自动微调，形成自演化、自调优、自管理的统一优化体；再借联邦通道汇聚海量语料，训练百亿级参数的超大 Graph Transformer，赋予其 Zero-shot 与 Few-shot 能力，实现跨 DBMS、跨领域、跨数据分布的通用泛化；最后让自演化优化器依托神经架构搜索与强化元学习，自主发现更优策略，配合自动特征工程突破人类经验天花板，打造越跑越聪明的终身演化循环。

参考文献

- [1] Yan Z, Uotila V, Lu J. Join Order Selection with Deep Reinforcement Learning: Fundamentals, Techniques, and Challenges[C]//Proceedings of the VLDB Endowment, 2023, 16(12): 3882-3885.
- [2] Wu Z, Negi P, Alizadeh M, et al. FactorJoin: A New Cardinality Estimation Framework for Join Queries[C]//Proceedings of the 2023 ACM SIGMOD International Conference on Management of Data. 2023: 1-19.
- [3] Li P, Wei W, Zhu R, et al. ALECE: An Attention-based Learned Cardinality Estimator for SPJ Queries on Dynamic Workloads[J]. Proceedings of the VLDB Endowment, 2023, 17(2): 197-210.
- [4] Chen T, Gao J, Chen H, et al. LOGER: A Learned Optimizer towards Generating Efficient and Robust Query Execution Plans[J]. Proceedings of the VLDB Endowment, 2023, 16(7): 1777-1789.
- [5] Zhu R, Chen W, Ding B, et al. Lero: A Learning-to-Rank Query Optimizer[J]. Proceedings of the VLDB Endowment, 2023, 16(6): 1466-1479.
- [6] Justen D, Ritter D, Fraser C, et al. POLAR: Adaptive and Non-invasive Join Order Selection via Plans of Least Resistance[J]. Proceedings of the VLDB Endowment, 2024, 17(6): 1350-1363.
- [7] Wang J, Trummer I, Kara A, et al. ADOPT: Adaptively Optimizing Attribute Orders for Worst-Case Optimal Join Algorithms via Reinforcement Learning[C]//Proceedings of the VLDB Endowment, 2023, 16(11): 3050-3063.
- [8] Marcus R, Papaemmanouil O. Deep Reinforcement Learning for Join Order Enumeration[C]//Proceedings of the First International Workshop on Exploiting Artificial Intelligence Techniques for Data Management (aiDM), 2018: 1-4.
- [9] Krishnan S, Yang Z, Goldberg K, et al. Learning to Optimize Join Queries with Deep Reinforcement Learning[J]. arXiv preprint arXiv:1808.03196, 2018.
- [10] Yu X, Li G, Chai C, et al. Reinforcement Learning with Tree-LSTM for Join Order Selection[C]//2020 IEEE 36th International Conference on Data Engineering (ICDE). IEEE, 2020: 1297-1308.
- [11] Chen J, Ye G, Zhao Y, et al. Efficient Join Order Selection Learning with Graph-based Representation[C]//Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining. 2022: 97-107.
- [12] Marcus R, Negi P, Mao H, et al. Neo: A Learned Query Optimizer[J]. Proceedings of the VLDB Endowment, 2019, 12(11): 1705-1718.
- [13] Marcus R, Negi P, Mao H, et al. Bao: Making Learned Query Optimization Practical[C]//Proceedings of the 2021 International Conference on Management of Data. 2021: 1275-1288.
- [14] Yang Z, Chiang W L, Luan S, et al. Balsa: Learning a Query Optimizer without Expert Demonstrations[C]//Proceedings of the 2022 International Conference on Management of Data.

2022: 931-944.

[15] Yu X, Chai C, Li G, et al. Cost-based or Learning-based? A Hybrid Query Optimizer for Query Plan Selection[J]. Proceedings of the VLDB Endowment, 2022, 15(13): 3924-3936.

[16] Trummer I, Wang J, Maram D, et al. SkinnerDB: Regret-bounded Query Evaluation via Reinforcement Learning[C]//Proceedings of the 2019 International Conference on Management of Data. 2019: 1153-1170.

[17] Zhang J. AlphaJoin: Join Order Selection à la AlphaGo[C]//PhD@ VLDB. 2020.

[18] Xu X, Zhao Z, Zhang T, et al. COOL: A Learning-to-Rank Approach for SQL Hint Recommendations[J]. arXiv preprint arXiv:2304.04407, 2023.